

Sprawozdanie z projektu 1

EasyAI

Joanna Wojnowska
Antonina Wąsikowska

16 marca 2026

Przedmiot: Inteligencja Obliczeniowa w Analizie Danych Cyfrowych

Prowadzący: Fabian Bogusz

Wybrana gra: Octospawn

1 Wstęp

Celem projektu było przygotowanie deterministycznego oraz probabilistycznego wariantu gry planszowej i porównanie algorytmów AI sterujących graczami. Porównano Negamax, Negamax z odcięciem alfa-beta oraz ExpectiMiniMax dla różnych głębokości przeszukiwania, analizując wpływ losowości na przebieg rozgrywki.

2 Opis gry

Octospawn jest rozszerzonym wariantem gry Hexapawn, rozgrywanym na planszy 4×4 przez dwóch graczy z czterema pionkami każdy.

2.1 Wariant deterministyczny

Gra jest przeznaczona dla dwóch graczy. Każdy gracz posiada cztery pionki ustawione w swoim początkowym rzędzie planszy. Pionki mogą poruszać się o jedno pole do przodu, jeśli pole jest wolne. Mogą również bić pionki przeciwnika po skosie w lewo lub w prawo. Pionki nie mogą się cofać.

Gracz wygrywa, gdy doprowadzi jeden ze swoich pionków do ostatniego rzędu planszy przeciwnika lub gdy przeciwnik nie ma żadnego możliwego ruchu.

2.2 Wariant probabilistyczny

W wariantcie probabilistycznym po każdym ruchu gracza istnieje prawdopodobieństwo (określone parametrem **chance** = 0.1), że jeden z pionków przeciwnika wcześniej zbitych przez aktualnego gracza zostanie przywrócony do gry. Wybrany pionek jest losowany z równą szansą spośród wszystkich zbitych pionków i wraca na swoje pierwotne pole startowe. Jeśli gracz nie zbił wcześniej żadnego pionka, zdarzenie losowe nie następuje.

3 Implementacja

3.1 Reprezentacja stanu gry

Gra została zaimplementowana w języku Python z wykorzystaniem biblioteki `easyAI`. Klasa `Octospawn` dziedziczy po klasie `TwoPlayerGame`. Pozycje pionków przechowywane są w słownikach, gdzie klucz oznacza identyfikator pionka, a wartość jego pozycję na planszy w postaci współrzędnych (*wiersz, kolumna*).

3.2 Funkcja oceny

Zastosowano heurystyczną funkcję oceny pozycji. W przypadku stanu przegranej zwracana jest silnie ujemna wartość -1000 . W pozostałych przypadkach ocena zależy od trzech czynników: różnicy liczby pionków, postępu pionków w kierunku linii końcowej oraz liczby dostępnych ruchów.

$$100 \cdot \Delta_{\text{pionki}} + 10 \cdot \Delta_{\text{postęp}} + 5 \cdot \text{liczbaruchów}$$

gdzie Δ oznacza różnicę między aktualnym graczem a przeciwnikiem.

4 Zastosowane algorytmy

4.1 Negamax

Algorytm Negamax został wykorzystany jako podstawowa metoda przeszukiwania drzewa gry. Opiera się on na założeniu, że interesy obu graczy są przeciwne, dlatego wartość pozycji dla jednego gracza jest przeciwna do wartości tej samej pozycji dla przeciwnika. Algorytm analizuje możliwe ruchy do określonej głębokości i wybiera ten, który maksymalizuje wartość pozycji.

4.2 Negamax z odcięciem alfa-beta

W celu przyspieszenia przeszukiwania zastosowano również wariant Negamax z odcięciem alfa-beta. Mechanizm ten pozwala pomijać analizę gałęzi drzewa, które nie mogą wpłynąć na końcowy wynik decyzji. Dzięki temu liczba analizowanych stanów gry jest mniejsza, co skraca czas działania algorytmu.

4.3 ExpectiMiniMax

W wariancie probabilistycznym po wykonaniu ruchu może wystąpić zdarzenie losowe (odrodzenie pionka). Z tego powodu zastosowano algorytm ExpectiMiniMax, który oprócz węzłów maksymalizujących i minimalizujących uwzględnia również węzły losowe. W implementacji zastosowano odcięcie alfa-beta w węzłach decyzyjnych, natomiast węzły losowe analizowano w pełni przez obliczanie wartości oczekiwanej.

Algorytm ExpectiMiniMax analizowano wyłącznie dla wariantu probabilistycznego, ponieważ w wariancie deterministycznym węzły losowe nie występują i algorytm jest tożsamy z klasycznym Negamax.

5 Metodologia eksperymentów

W celu porównania algorytmów przeprowadzono serię automatycznych rozgrywek pomiędzy dwoma agentami AI. W każdej serii rozegrano 100 gier, a gracz rozpoczynający był zmieniany naprzemiennie, aby ograniczyć wpływ przewagi pierwszego ruchu.

Podczas eksperymentów mierzono:

- liczbę zwycięstw każdego z graczy (z podziałem na gracza rozpoczynającego i drugiego),
- średni czas wyboru pojedynczego ruchu przez algorytm.

Czas mierzono osobno dla każdego wywołania algorytmu, a następnie uśredniano po wszystkich ruchach w partii i po wszystkich partiach w serii. W przeprowadzonych eksperymentach nie zaobserwowano remisów, ponieważ nie były definiowane w regułach gry.

Przeprowadzono dwa rodzaje eksperymentów: symetryczne (ten sam algorytm po obu stronach) oraz asymetryczne (różne algorytmy grać przeciwko sobie). Parametry: głębokość 3 i 7, prawdopodobieństwo przywrócenia pionka ($prob \in \{0.0, 0.1\}$), algorytmy: Negamax (z/bez alfa-beta) oraz ExpectiMiniMax.

6 Wyniki eksperymentów

Tabela 1: Wyniki rozegranych eksperymentów

Algorytm i konfiguracja	P1	P2	Zaczynający	Drugi
Negamax, depth=3, alfa-beta, prob=0.1	47	53	81	19
Negamax, depth=3, bez alfa-beta, prob=0.1	49	51	79	21
Negamax, depth=7, alfa-beta, prob=0.1	50	50	78	22
Negamax, depth=7, bez alfa-beta, prob=0.1	47	53	77	23
ExpectiMiniMax, depth=3, prob=0.1	52	48	90	10
ExpectiMiniMax, depth=7, prob=0.1	50	50	78	22
Negamax, depth=3, alfa-beta, prob=0.0	50	50	100	0
Negamax, depth=3, bez alfa-beta, prob=0.0	50	50	100	0
Negamax, depth=7, alfa-beta, prob=0.0	50	50	100	0
Negamax, depth=7, bez alfa-beta, prob=0.0	50	50	100	0

Kolumny *Zaczynający/Drugi* odnoszą się do kolejności pierwszego ruchu w danej partii.

Tabela 2: Bezpośrednie starcia algorytmów (prawdopodobieństwo=0.1, 100 gier)

P1 vs P2	P1	P2	Zaczynający	Drugi
Negamax(d=3,ab) vs Negamax(d=7,ab)	39	61	87	13
Negamax(d=3,ab) vs ExpectiMiniMax(d=3)	47	53	77	23
Negamax(d=7,ab) vs ExpectiMiniMax(d=7)	40	60	84	16

Tabela 3: Średni czas decyzji

Algorytm i konfiguracja	Deterministyczna [s]	Probabilistyczna [s]
Negamax, depth=3, alfa-beta	0.0008	0.0007
Negamax, depth=3, bez alfa-beta	0.0011	0.0010
Negamax, depth=7, alfa-beta	0.0070	0.0120
Negamax, depth=7, bez alfa-beta	0.1036	0.1187
ExpectiMiniMax, depth=3	—	0.0099
ExpectiMiniMax, depth=7	—	1.2304

Algorytm ExpectiMiniMax został uruchomiony jedynie dla wariantu probabilistycznego. W przypadku wariantu deterministycznego ($prob = 0.0$) drzewo gry nie zawiera węzłów losowych.

7 Analiza wyników

Na podstawie uzyskanych wyników można sformułować następujące obserwacje:

- W wariancie deterministycznym gracz rozpoczynający wygrywał 100% partii we wszystkich konfiguracjach, co wskazuje na bardzo silną przewagę pierwszego ruchu w tej grze. Wynik 50 : 50 wynika wyłącznie z naprzemiennego zmieniania gracza rozpoczynającego.
- W wariancie probabilistycznym przewaga gracza rozpoczynającego nadal była wyraźna, choć mniejsza niż w wariancie deterministycznym. W zależności od konfiguracji gracz rozpoczynający wygrywał od 77 do 90 partii na 100. Losowość zmniejsza wpływ pierwszego ruchu, ale go nie eliminuje.
- Zwiększenie głębokości przeszukiwania z 3 do 7 zmniejszało przewagę gracza rozpoczynającego w wariancie probabilistycznym. Głębsze przeszukiwanie pozwala drugiemu graczowi lepiej wykorzystywać szanse wynikające z losowości, co częściowo rekompensowało przewagę pierwszego ruchu.
- Zastosowanie odcięcia alfa-beta nie wpływało istotnie na skuteczność algorytmu Negamax, jednak znacząco poprawiało jego wydajność obliczeniową. Dla głębokości 7 w wariancie probabilistycznym średni czas wyboru ruchu bez odcięcia wynosił 0.1187s, a z odcięciem jedynie 0.0120s — około dziesięciokrotne przyspieszenie przy zachowaniu podobnej jakości decyzji.
- Algorytm ExpectiMiniMax był wyraźnie wolniejszy od Negamax. Dla głębokości 3 średni czas wynosił 0.0099s, natomiast dla głębokości 7 aż 1.2304s. Wynika to z konieczności uwzględniania węzłów losowych i obliczania wartości oczekiwanej w drzewie gry.
- W bezpośrednich grach między różnymi algorytmami większa głębokość dawała przewagę: Negamax(d=7) wygrywał 61 do 39 z Negamax(d=3). ExpectiMiniMax wygrywał z Negamax zarówno przy głębokości 3, jak i przy głębokości 7, co potwierdza, że jawne modelowanie losowości poprawia jakość decyzji w wariancie probabilistycznym niezależnie od głębokości przeszukiwania.

8 Napotkane problemy

Podczas realizacji projektu napotkano kilka problemów implementacyjnych. Klasyczny algorytm Negamax nie jest naturalnie przystosowany do obsługi zdarzeń losowych — w wariancie probabilistycznym wymaga to dodatkowych założeń upraszczających, natomiast pełniejsze modelowanie losowości wymaga użycia algorytmu ExpectiMiniMax.

Implementacja ExpectiMiniMax okazała się wyraźnie bardziej kosztowna obliczeniowo niż Negamax, szczególnie dla większej głębokości przeszukiwania. Dodatkowym wyzwaniem było zachowanie spójności pomiędzy właściwą mechaniką gry (z losowością) a uproszczonym modelem używanym przez algorytm do analizy przyszłych stanów. W szczególności poprawne przypisanie referencji gracza po zamianie tur wymagało szczególnej uwagi.

9 Wnioski

Przeprowadzone eksperymenty pokazały, że w wariancie deterministycznym gry Octospawn występuje bardzo silna przewaga pierwszego ruchu, gracz rozpoczynający rozgrywkę zawsze wygrywa przy odpowiednich ruchach. Wprowadzenie losowości zmniejsza tę przewagę, ale jej nie eliminuje. Z praktycznego punktu widzenia Negamax z odcięciem alfa-beta okazał się najlepszym kompromisem między jakością decyzji a czasem obliczeń. ExpectiMiniMax lepiej odpowiada charakterowi gry probabilistycznej, ponieważ jawnie modeluje zdarzenia losowe, jednak odbywa się to kosztem wyraźnie większego czasu działania. Bezpośrednie starcia algorytmów potwierdziły, że ExpectiMiniMax podejmuje lepsze decyzje w wariancie probabilistycznym niż Negamax przy tej samej głębokości przeszukiwania.